

HTTP - a Hybrid Transformer for Trajectory Predictor

Arico Amaury
BruFace student

Colot Emmeran
BruFace student

Lungu Loric
BruFace student

I. INTRODUCTION

Autonomous vehicles are widely considered to be a major part of the future of transportation. However, their deployment still faces important challenges, particularly in the detection and motion prediction of surrounding agents such as pedestrians and vehicles within complex driving environments.

Developing such systems requires both large-scale multimodal datasets collected from multiple sensors and robust machine learning models capable of understanding dynamic traffic scenes. In the context of this project, our group focused specifically on vehicle trajectory prediction in urban environments.

The obtained results are consistent with the existing literature on the subject. The following sections dive deeper into the different aspects of the project, including the dataset, model architecture, training procedure, and evaluation metrics.

II. STATE OF THE ART

This section presents the main concepts and architectures related to multimodal perception and trajectory forecasting for autonomous driving systems.

A. NuScenes Dataset

For this project, we relied on the nuScenes dataset [1], which consists of driving scenes recorded using a Renault Zoe electric vehicle equipped with several type of sensors, including cameras, lidar sensors, and radar sensors.

The dataset contains recordings collected in Boston and Singapore using a similar sensor configuration. In this work, we use the mini version of the original dataset containing 1000 scenes of 20 seconds recorded at a frequency of 2 Hz.

1) *Objective of the nuScenes dataset:* Reliable detection and tracking of surrounding agents are essential components of autonomous driving systems. To safely navigate complex urban environments, autonomous vehicles must continuously estimate the position and behavior of nearby road users such as pedestrians, cyclists, and vehicles.

Machine learning methods now play a central role in these perception tasks. However, although standard image datasets are highly accessible, they fail to fully capture the complexity of autonomous driving environments. As a result, multimodal datasets are generally preferred despite being significantly

more difficult and expensive to collect.

2) *Multimodal datasets:* No individual sensor can independently provide all the information required for reliable autonomous driving. For this reason, modern autonomous vehicles combine several complementary sensing technologies.

Cameras

- Advantage: cameras provide detailed visual information such as colors, textures, edges, and lighting conditions, enabling efficient object classification and accurate 2D perception.
- Disadvantage: cameras provide limited direct depth estimation and reduced accuracy for precise 3D localization.

Lidar point clouds

- Advantage: lidar sensors provide highly accurate 3D localization of surrounding objects.
- Disadvantage: lidar measurements are sparse and contain less semantic information than image data. Their detection range is also generally limited between 50 to 150 meters.

Radar sensors

- Advantage: radar sensors provide an extended detection range, typically between 200 and 300 meters, it also enables velocity estimation using Doppler effect.
- Disadvantage: radar measurements are even sparser and less spatially precise than lidar observations.

Because these type of sensors provide complementary information, combining them generally improves the robustness and reliability of perception systems in complex traffic environments.

3) *Annotations:* Objects present in the keyframes are manually annotated and divided into 23 semantic categories, including several vehicle classes, pedestrians, cyclists, and additional road users.

B. Decoupling Motion Forecasting

DeMo [6] proposes a trajectory forecasting framework for autonomous driving that aims to improve both the diversity and temporal consistency of predicted trajectories. Most recent forecasting methods rely on vectorized scene representations and follow a one-query-one-trajectory strategy, where each query directly predicts a single future trajectory. Although effective, this approach often provides limited modeling of the temporal evolution of vehicle states.

To overcome this limitation, DeMo introduces a decoupled query representation combined with State Space Models based on the Mamba architecture. Compared to conventional Transformer-based sequence modeling approaches, Mamba offers improved efficiency for long sequential dependencies while preserving strong temporal modeling capabilities.

The proposed framework separates trajectory representation into two complementary query types:

- **Motion mode queries**, which encode possible driving intentions such as lane following, left turns, or right turns.
- **Dynamic state queries**, which model the evolution of vehicle states over time and enforce temporal consistency across predicted trajectories.

The scene information is first extracted from HD maps and neighboring agents. Map polylines are encoded using a PointNet-based encoder, while historical trajectories are processed using several unidirectional Mamba blocks to capture temporal dependencies. These representations are then combined to build the final scene context features.

To jointly model motion intentions and temporal dynamics, the framework introduces three complementary modules:

- **State Consistency Module**: this module processes the dynamic state queries. The queries are first initialized through a Multi-Layer Perceptron (MLP), then refined through cross-attention interactions with the scene context features. Bidirectional Mamba layers are subsequently used to model temporal relationships between successive dynamic states.
- **Mode Localization Module**: this component focuses on directional intentions. Motion mode queries interact with the scene context through multi-head attention layers before MLP blocks estimate future trajectories and their associated confidence scores.
- **Hybrid Coupling Module**: this module combines motion mode queries and dynamic state queries in order to jointly model spatial intentions and temporal dynamics. Attention mechanisms are used to exchange information between the different query types before generating the final trajectory predictions.

The model is trained using a global loss function combining trajectory regression, classification losses, and auxiliary losses related to motion modes and temporal state consistency.

By separating directional intentions from temporal dynamic modeling, DeMo achieves a more detailed representation of future trajectories.

Experimental results obtained on the nuScenes and Argoverse 2 datasets show that this decoupled strategy improves trajectory prediction accuracy while maintaining efficient inference performance. This approach will serve as one of the main inspirations for our proposed model.

III. OUR MODEL

The HTTP model presented in this paper uses both LiDAR points and pictures taken from the front camera in order to predict the trajectory of nearby vehicles. The inputs are refined

using either the PointPillar model [4] for the LiDAR, or the ResNet-18 model [3] to process the pictures.

The extracted features are then fused inside a Transformer that produces a trajectory forecast over a horizon of 6 seconds.

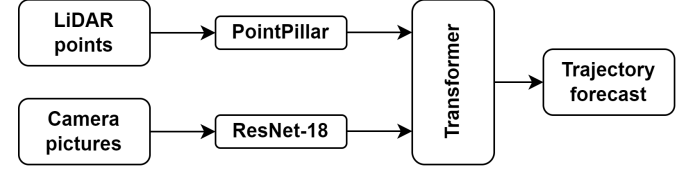


Fig. 1: General structure of the HTTP model.

Figure 1 showcases a block structure of the model and its different components are detailed hereafter.

A. PointPillar

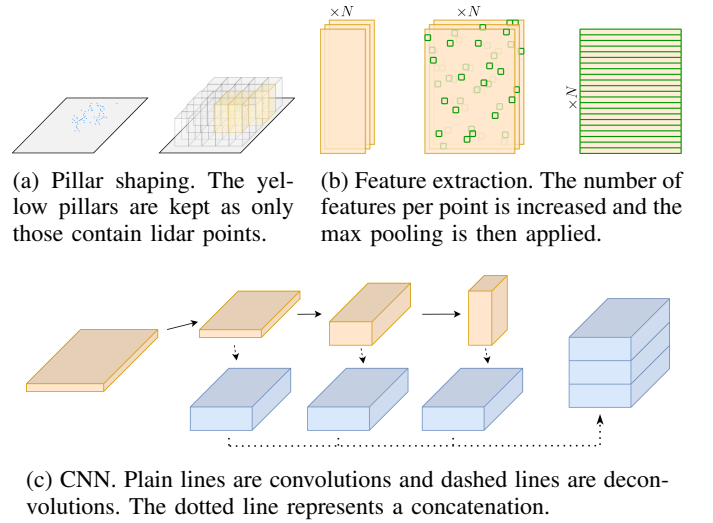


Fig. 2: PointPillar model overview.

The feature extraction of the PointPillar model is split into three successive stages, all shown in fig. 2.

The LiDAR points are first stacked in pillars based on their coordinates. A pillar contains at most 32 points and empty pillars are removed to reduce the memory footprint.

The second stage extracts features from the voxels by adding to each voxel its relative position to the pillar center and the pillar centroid. The number of features is then increased by passing through a two layers network, consisting of a linear operation followed by a batchnorm and a ReLU. Only the maximal feature over the voxels is then kept, leading to 64 features per pillar.

The last stage is a CNN, performing convolutions and deconvolutions on the pillars to extract data dependencies at different resolutions. Note that the pillars from the last stage are first placed on a grid matching the original position of the pillars.

The dimensions of the map on which pillars are placed were determined in order to contain all the voxels in the dataset.

B. Resnet-18

Rather than projecting the raw high-dimensional camera data directly into latent feature space at the encoder, the input are processed through a feature extraction network introduced in “Deep Residual Learning for Image Recognition”, known as **Resnet**. Historically, in 2015, image treatment was led by the assumption that increasing network depth steadily improves the feature extraction capacity of the network. However, this idea hits severe optimization bottlenecks. Beyond the well-known vanishing/exploding gradient problem during the backpropagation of deep network, the model encounters a saturation and finally a degradation of the training error once reaching a threshold. The latter degradation reveals a fundamental limitation of such deep network stacking linear transformation and non-linear activation function (e.g., ReLu) which is expressed, in the literature, by the difficulty of performing an identity mapping. This bottleneck translates the inability for the deep neural network to perform an identity transformation due to their design. To give an example, if a 30 layers Convolutional Neural Network (CNN) training leads to a lower error than a 50 layers counterpart, it implies that the 20 additional layers are, in the first place, unable to converge towards the same model accuracy, but furthermore, are distorting the output features by the inability of the NN layers to operate an identity transformation. To overcome these limiting performance constraints, **Resnet** introduces a direct path between the embedded input vector and the output of the feedforward performing a elementwise summation - fig. 3. This path, also known as shortcut connection, makes the construction of an identity mapping trivial (no weight paramater through the direct connection), while optimizing the features of the **input residual** - $\mathcal{F}(x)$.

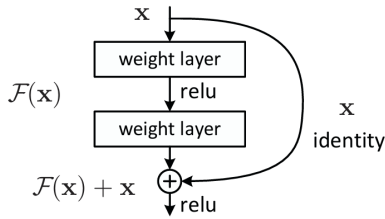


Fig. 3: Resnet block: residual learning.

The HTTP model incorporates a Resnet-18 architecture for image processing. As indicated by the naming, the network contains a total of **18** layers with learnable parameters. The image is first resized, normalized and centered around zero prior to being propagated through the Resnet backbone. The output features are mapped via a linear layer and dimensioned to match the target space required for the embedding vectors at the transformer encoder. Figure 4 showcases the network details.

It is worth noting that while the Resnet-18 has been built from scratch in a dedicated class for learning purpose, the model utilizes the standardized Resnet library enabling the

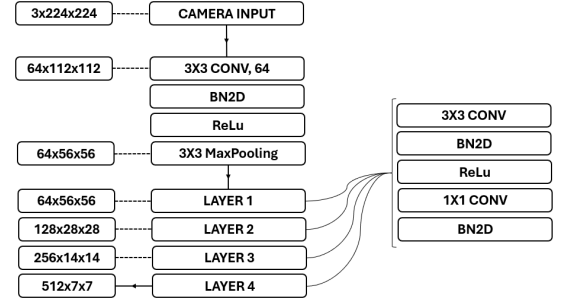


Fig. 4: Resnet18 network architecture.

integration of pre-trained weights reducing the training convergence time.

C. Long Short Term Memory

The remaining critical data at our disposal is the historical driving trajectory of the target vehicle. This coordinate sequence carries critical information about the kinetic and momentum state of the vehicle. Nevertheless, these measurements are time-dependent and a standard feedforward linear NN is poorly suited to extract features spanning across distinct timesteps. To efficiently extract these dynamics, our model introduces a Recurrent Neural Network block (RNN), and more specifically a Long Short Term Memory (LSTM) block - [5]. Unlike to a standard RNN relying on a single memory state (known as the hidden state, h_i), the LSTM decouples memory representation into a long term memory cell (C_i) and a short term memory state (h_i).

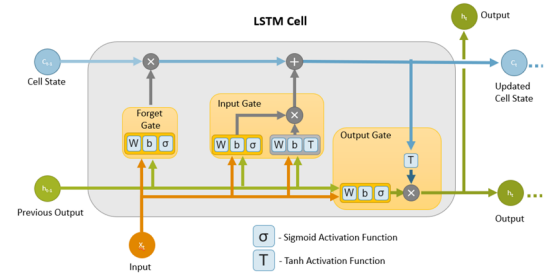


Fig. 5: LSTM network architecture.

The long term memory cell is linked with the short term memory state and the current input through **three gates**, namely the forget gate, the input gate and the candidate gate. These gates regulate how the input and the short term memory modulate the long term memory cell, C_t . It is achieved either by attenuating features by element-wise multiplication with the forgetting output, f_t or by adding new features through the multiplication of the candidate gate’s output, $\tilde{c}_t$ and input gate’s output, m_t . The short term memory at the next timestep is finally computed by the multiplication of the long term memory cell, at time instant $t - 1$ and the output gate vector, o_t .

$$\begin{aligned}
f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) & (1) \\
\tilde{c}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) & (2) \\
m_t &= \sigma(W_m \cdot [h_{t-1}, x_t] + b_m) & (3) \\
\mathcal{C}_t &= \mathcal{C}_{t-1} \odot f_t + \tilde{c}_t \odot m_t & (4) \\
o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) & (5)
\end{aligned}$$

$W_h, W_x, W_f, W_c, W_m, W_o$ and b_f, b_c, b_m, b_o denote respectively the weight matrices and the bias vectors associated to the gates, h_{t-1} the previous hidden state and x_t the current input.

The primary advantage of such architecture lies in its stability against the vanishing/exploding gradient descent which is caused, for a standard RNN, by the recursive update of the hidden state at each time step making this long-term temporal dependency sensitive to such gradient problematic.

For the standard RNN, The loss gradient of the hidden state is computed by repeated weight matrix multiplications, W_h , leading to a vanishing or exploding gradient depending on the largest matrix eigenvalue value.

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad (6)$$

$$\frac{\partial \varepsilon}{\partial h_k} = \frac{\partial \varepsilon}{\partial h_t} \cdot \prod_{j=k+1}^t \text{term}(\tanh(\alpha)) \cdot W_h^T \quad (7)$$

W_h and W_x denote the weight matrices associated to the hidden state and the input, b the bias, ε the error loss and α the term inside the function $\tanh$, eq. (6).

In case of LSTM block, the gradient computation is significantly different and depends on the dynamic activation vector (f_j) programmed, or trained, by the network. This vector varies in function of the current input and the previous hidden state injected in the block.

$$\frac{\partial \varepsilon}{\partial \mathcal{C}_k} = \frac{\partial \varepsilon}{\partial \mathcal{C}_t} \cdot \prod_{j=k+1}^t \frac{\partial \mathcal{C}_t}{\partial \mathcal{C}_{t-1}} \quad (8)$$

$$\frac{\partial \mathcal{C}_t}{\partial \mathcal{C}_{t-1}} \approx f_t \quad (9)$$

Regarding our model implementation, upstream of the LSTM block, the original past positional history input (4 frames $\times$ 2 coordinates) is first processed by a 2-layer Linear network projecting the 2 coordinates features into 64 then 256 dimensions space. The initial hidden state h_0 and cell state $\mathcal{C}_0$ are initialized as zero vectors. Then, the LSTM block is instantiated by the *CustomLSTMCell* class. To optimize computational efficiency and maximize GPU utilization, the weight matrices for all four internal gates are vertically stacked into an unified linear transformation block. Following the sequential execution of the gate equations, the LSTM block returns a post-processed hidden state vector output consisting of 256 features.

D. Transformer

The final module and core of our multi-modal model is the encoder-decoder Transformer. As briefly introduced in the state of the art section, the main idea of the transformer is directly inspired from the framework proposed in *DeMo: Decoupling Motion Forecasting into Directional Intentions and Dynamic States* - [6]. Specifically, we adopt the core concept of decoupling the decoder queries into two distinct components: *intent queries*, also referred to as motion or mode queries, mapping the different possible trajectories and the *dynamic queries* inferring the instantaneous momentum state of the target vehicle, which are outputted by our LSTM block. Within the framework of our project, the output of the LSTM module generates a single dynamic query spanning the space $\mathbb{R}^{1 \times 256}$.

At the encoder stage, the lidar points and camera pixels are projected into the unified encoder features space dimensions after being pre-processed respectively through the *Pointpillar* and the *Resnet-18* backbones introduced in the previous subsections. The attention head module being permutation invariant (the head is not a sequential process and treat all the tokens in parallel), the network is unaware of the spatial distribution of the tokens and must therefore be explicitly injected into the input embedding vectors. To do so, our model encodes the structural positions of the processed LiDAR and camera outputs using a 2D Sinusoidal Positional Encoding, which are summed with the projected data to form the final attention head inputs.

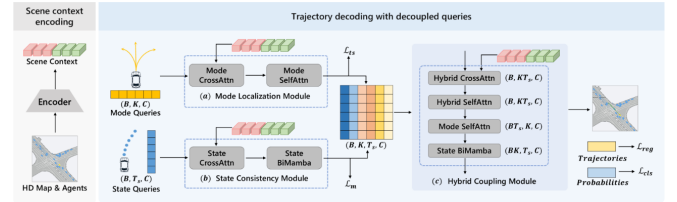


Fig. 6: DeMo model - Transformer architecture.

To meet project deadlines and maintain an appropriate architectural scope for this project, our group opted to reduce the complexity of the Transformer decoder proposed in the DeMo paper - which utilizes two parallel distinct decoders with state and motion queries as input queries finalized with a third fused multi-head attention decoder - Figure 6. While we preserved the core query decoupling strategy, the transformer decoder has been simplified by adapting the standard network structure described in "End-to-End Object Detection with Transformers" - [2] - Figure 7.

The resulting decoder is constructed as follows: the decoder queries are formed by summing the state query, outputted by the LSTM network which represents the vehicle state content, and the intent queries (M modes), initialized as distinct random vectors, matching the target features dimension, $\mathbb{R}^{M \times 256}$. These intent queries are learnable parameters inferring the spatial intent of the vehicle's future heading. The combined input vector lies in the space $\mathbb{R}^{M \times 256}$. The output of the

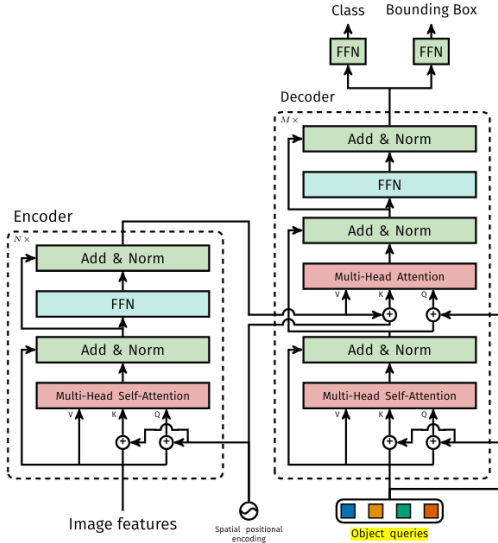


Fig. 7: DETR model - Transformer architecture.

decoder is, in fine, processed by a multi-modal trajectory prediction layer projecting the decoded representations into three distinct parallel heads: the future trajectory space, $\mathbb{R}^{M \times 12 \times 2}$ (predicting X and Y displacements over 12 future steps), the vehicle heading angle space (providing the vehicle orientation across the time horizon), $\mathbb{R}^{M \times 12 \times 1}$, and the motion mode probability space, $\mathbb{R}^{M \times 1}$.

It is worth noting that the DETR framework investigated the most suitable spatial positioning encoding at the encoder and the decoder stages - Figure 8. In an attempt to reproduce the investigated benefit, we incorporated DETR spatial encoding strategy but did not bring any improvements on the metrics and/or slightly worsening the trajectory prediction for some scene configuration. Nevertheless, we made the choice to keep the 2D spatial encoding at the input of the attention heads following the proposed transformer architecture.

spatial pos. enc. encoder	spatial pos. enc. decoder	output pos. enc. decoder	AP	Δ	AP ₅₀	Δ
none	none	learned at input	32.8	-7.8	55.2	-6.5
sine at input	sine at input	learned at input	39.2	-1.4	60.0	-1.6
learned at attn.	learned at attn.	learned at attn.	39.6	-1.0	60.7	-0.9
none	sine at attn.	learned at attn.	39.3	-1.3	60.3	-1.4
sine at attn.	sine at attn.	learned at attn.	40.6	-	61.6	-

Fig. 8: DETR model - Spatial encoding performance.

Lastly, we adopt the cost function strategy proposed by the *DeMo* framework. The total objective combines a trajectory regression loss, $\mathcal{L}_{\text{reg}}$, a heading state cost, $\mathcal{L}_{\text{state}}$, and a multi-modal classification loss, $\mathcal{L}_{\text{cls}}$. The regression term computes the L_1 norm of the displacement error, scaled by a time-dependent weight parameter that increases exponentially with the future time horizon index to prioritize long-term accuracy. The state cost, $\mathcal{L}_{\text{state}}$, penalizes the absolute heading angle error, normalized between 0 and 1. Finally, the classification loss, $\mathcal{L}_{\text{cls}}$, is formulated as a binary cross-entropy metric

evaluated with soft targets, y_k . The utilization of a soft target was motivated by the produced results but also by allowing the gradients to propagate through all trajectory modes simultaneously, preventing a winner-take-all collapse of the non-winning motions.

$$\mathcal{L}_{\text{reg}} = \frac{1}{T} \sum_{t=1}^T \left(e^{\frac{t}{T}} \cdot |\hat{x}_t - x_t| \right) \quad (10)$$

$$y_k = \exp \left(-\frac{\text{FDE}_k}{\sigma_{\text{variance}}} \right) \quad (11)$$

$$\mathcal{L}_{\text{cls}} = -\frac{1}{K} \sum_{k=1}^K \left[y_k \log(\sigma(x_k)) + (1 - y_k) \log(1 - \sigma(x_k)) \right] \quad (12)$$

$$\mathcal{L} = \mathcal{L}_{\text{reg}} + w_1 \cdot \mathcal{L}_{\text{s}} + w_2 \cdot \mathcal{L}_{\text{cls}} \quad (13)$$

w_1 and w_2 denotes the weights parameters respectively fixed empirically to 1.7 and 1.0.

E. Dataset

The model training and validation requires a dedicated dataset of vehicles to track. To do so, the *Nuscenes* dataset was scanned and filtered to only retain vehicles present in **4** past and **12** futures scene frames. Fortunately for us, Nuscenes API supports the user into dividing the scenes into training and validation scenes set - *create_splits_scenes* API enables a split of the scenes by scene name sorting. The vehicles and scene name filtering was executed in the data loader initialization.

F. Training

The proposed model is trained over 12 epochs, as empirical observations indicated that extending training beyond this horizon led to overfitting. Initially, the model was trained to predict 6 future trajectories without spatial positional encoding resulting to encouraging results - see details in *Results section*. The multi-modal output was afterwards expanded to 10 trajectories motions. This output space expansion was required to compare our model performance and the current state-of-the-art results detailed in the *DeMo* paper. Throughout the training, the batch size was maintained to 1. This constraint arises from the variable size of the raw LiDAR data resulting into variable pillar tensors size at the output of the *PointPillars* process. a zero-padding of the tensors ensuring fixed dimension size would enable mini-batch training through parallelization.

G. Inference and Accuracy Metrics

To evaluate the model performances, we relied on the following metrics :

1) **MinADE** : ADE_i , Average Displacement Error represents the average distance between the predicted trajectory k and the ground truth over a time horizon T for an agent i . If K trajectory hypotheses are predicted, minADE_i corresponds to the trajectory k whose ADE_i is minimal.

$$\min\text{ADE}_{i,K}(t^*) = \min_{k \in K} \frac{1}{T} \sum_{t=t^*}^{t^*+T} \|\hat{p}_i^k(t) - p_i(t)\| \quad (14)$$

This can be average over all the N agents tracked to give the overall minADE:

$$\min\text{ADE}_K = \frac{1}{N} \sum_{i=1}^N \min\text{ADE}_{i,K} \quad (15)$$

2) **MinFDE** : the minimum Final Displacement Error is computed in a similar manner to minADE but only considers the final time step $T + t^*$:

$$\min\text{FDE}_{i,K}(t^*) = \min_{k \in K} \|\hat{p}_i^k(t^* + T) - p_i(t^* + T)\| \quad (16)$$

$$\min\text{FDE}_K = \frac{1}{N} \sum_{i=1}^N \min\text{FDE}_{i,K} \quad (17)$$

3) **Miss rate** : this metric measures the percentage of missed predictions meaning the percentage of agents for which the minFDE is above a threshold distance d :

$$\text{Missed prediction for agent } i = \min\text{FDE}_i > d \quad (18)$$

$$\text{MR} = \frac{\text{missed predictions}}{\text{number of predictions}} \quad (19)$$

The total number of predictions corresponds to the number of evaluated agents. The distance d was fixed at 2m.

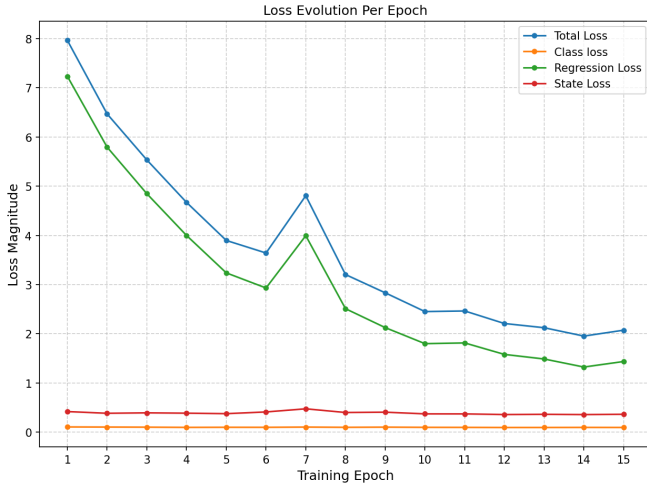


Fig. 9: Evolution of the loss during training. This was computed with 6 possible trajectories.

To determine the number of epochs for training, two of the metrics were used:

Their respective evolution with the number of epochs is plotted on fig. 10 and the performances start to stagnate from

epoch 8, with close to no improvement with an additional training. The model used is therefore the one at the 15th epoch, according to the increase in training loss seen in fig. 9.

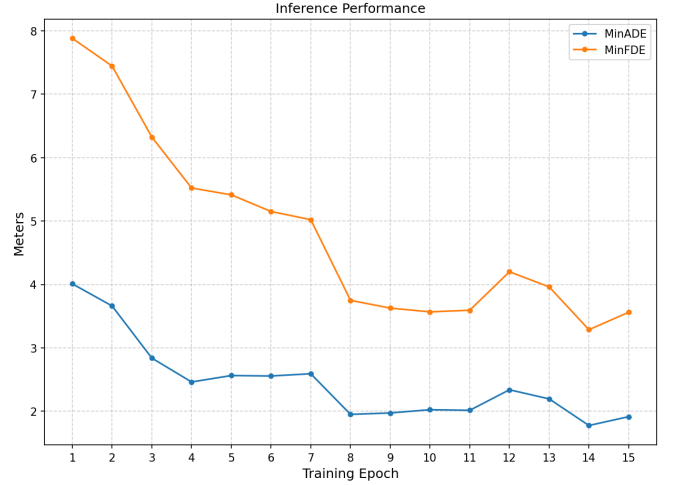


Fig. 10: Evolution of the performances during training. This was computed with 6 possible trajectories.

Note that some metrics of table I required 10 different trajectories motion and the number of epoch was there limited to 12 for similar reasons.

IV. RESULTS

Figure 11 shows how HTTP predicts the trajectory in different scenarios. With moving cars, the model is quite accurate at least for the first few seconds, as seen in figs. 11a and 11d.

Stopped cars are much more difficult to handle, as the momentum of the cars is taken into account, making it really hard to understand that a car might start moving. Figure 11b is therefore unable to accurately predict the trajectory of the car that was just stopped. A way to improve this would be to use more situations with traffic lights in the training dataset, or to implement a specific loss function for areas where car have to stop.

Parked cars are accurately predicted, as shown on fig. 11c.

Table I compares our HTTP model with the state of the art, showing our model is a bit behind the best models. The miss rates are suspiciously good and is maybe due to the limited size of our dataset.

V. CONCLUSIONS

This project investigated vehicle trajectory prediction for autonomous driving using multimodal data from the nuScenes dataset.

A hybrid architecture named HTTP was developed by combining LiDAR point clouds processed with PointPillar, camera images encoded through ResNet-18, and vehicle trajectory history modeled using an LSTM network. These features were then fused within a Transformer architecture inspired by the query decoupling strategy proposed in the DeMo framework.

TABLE I: Performance comparison between HTTP and state of the art models. Data taken form [6]. The index on the performances indicates the number of different of forecasted trajectories.

Method	$minFDE_1$	$minADE_5$	$minADE_{10}$	MR_5	MR_{10}
Trajectron++	9.52	1.88	1.51	0.70	0.57
LaPred	8.37	1.47	1.12	0.53	0.46
P2T	10.50	1.45	1.16	0.64	0.46
GOHOME	6.99	1.42	1.15	0.57	0.47
CASPNet	-	1.41	1.19	0.60	0.43
Autobot	8.19	1.37	1.03	0.62	0.44
THOMAS	6.71	1.33	1.04	0.55	0.42
PGP	7.17	1.27	0.94	0.52	0.34
LAformer	6.95	1.19	1.19	0.48	0.48
DeMo	6.60	1.22	0.89	0.43	0.34
HTTP (ours)	9.02	2.04	1.99	0.18	0.18

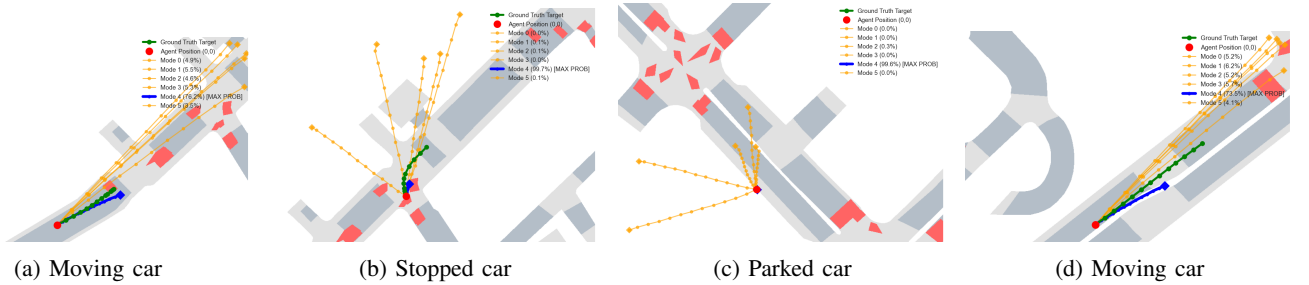


Fig. 11: Overlay of the true trajectory (green) of the cars and the predicted trajectory (blue). The yellow lines correspond to trajectories not selected by the model. Red areas are places where cars should stop.

The obtained results demonstrate that the proposed architecture is capable of producing realistic trajectory predictions in a variety of driving scenarios. On the validation dataset, the model achieved a minADE of 2.04 m, a minFDE of 3.68 m and a miss rate of 18.4%. Qualitative results showed good performance for moving and parked vehicles, while stopped vehicles remain more challenging to predict.

Although the proposed model does not reach the performance of recent state-of-the-art methods, it successfully demonstrates the benefits of combining multimodal perception and Transformer-based trajectory forecasting. Future improvements could include richer map information, a more advanced decoder architecture and dedicated mechanisms for handling traffic lights and other traffic constraints.

Overall, this work highlights both the potential and the challenges of multimodal trajectory prediction for autonomous driving applications.

REFERENCES

- [1] Holger Caesar et al. “nuScenes: A multimodal dataset for autonomous driving”. In: *CVPR*. 2020.
- [2] Nicolas Carion et al. “End-to-End Object Detection with Transformers”. In: *CoRR* abs/2005.12872 (2020). arXiv: [2005.12872](https://arxiv.org/abs/2005.12872). URL: <https://arxiv.org/abs/2005.12872>.
- [3] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- [4] Alex H. Lang et al. “PointPillars: Fast Encoders for Object Detection from Point Clouds”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 12697–12705. DOI: [10.1109/CVPR.2019.01298](https://doi.org/10.1109/CVPR.2019.01298).
- [5] Christian Bakke Vennerød, Adrian Kjærø, and Erling Stray Bugge. “Long Short-term Memory RNN”. In: *CoRR* abs/2105.06756 (2021). arXiv: [2105.06756](https://arxiv.org/abs/2105.06756). URL: <https://arxiv.org/abs/2105.06756>.
- [6] Bozhou Zhang, Nan Song, and Li Zhang. *DeMo: Decoupling Motion Forecasting into Directional Intentions and Dynamic States*. 2024. arXiv: [2410.05982](https://arxiv.org/abs/2410.05982) [cs.CV]. URL: <https://arxiv.org/abs/2410.05982>.